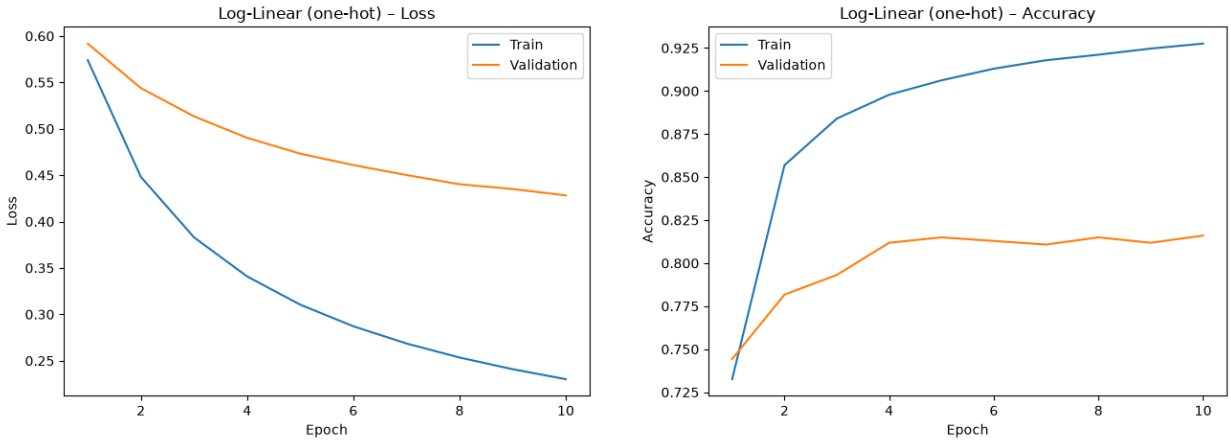
Natural Language Processing - Ex5

Sentiment Analysis with Neural Models

Nurit Tolkowsky and Shaul Tolkowsky

We trained all four models on the Stanford Sentiment Treebank with the hyperparameters required by the exercise. The three full-data models (Sections 6–8) were trained on the full sentences together with their sub-phrases (73,570 training items); the fine-tuned Transformer (Section 9) was trained on 2,500 randomly sampled full sentences only. For each model we report the train/validation loss and accuracy curves, the final test loss and accuracy, and the accuracy on the two special test subsets (62 negated-polarity sentences and 50 rare-word sentences).

Section 6 - Simple Log-Linear Model (one-hot)



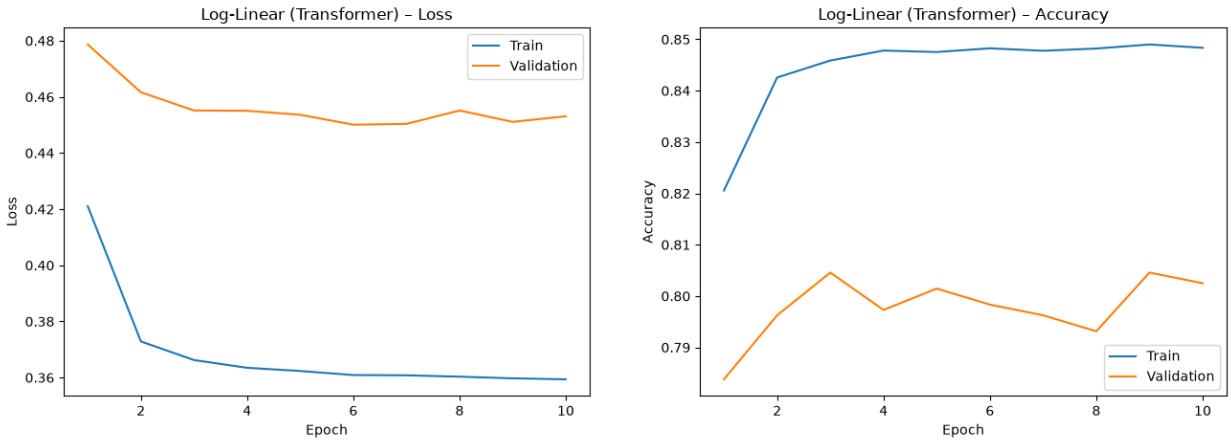
(a) Train/validation loss

(b) Train/validation accuracy

Figure 1: Section 6 - one-hot log-linear.

Quantity	Value
Test loss	0.4237
Test accuracy	0.8233
Negated-polarity accuracy (62 ex.)	0.6452
Rare-words accuracy (50 ex.)	0.5400

Section 7 - Transformer-Embedding Log-Linear Model



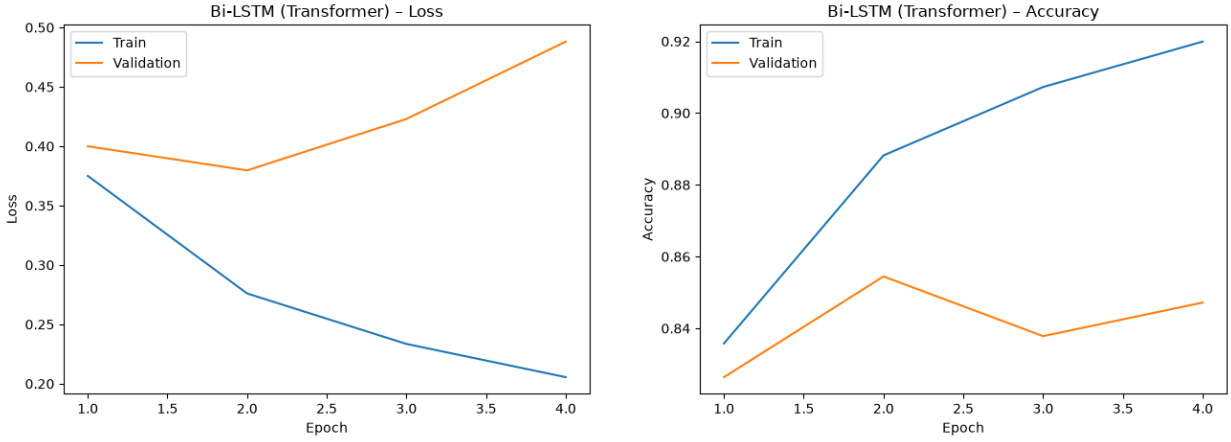
(a) Train/validation loss

(b) Train/validation accuracy

Figure 2: Section 7 - Transformer-embedding log-linear.

Quantity	Value
Test loss	0.3967
Test accuracy	0.8368
Negated-polarity accuracy (62 ex.)	0.6290
Rare-words accuracy (50 ex.)	0.8400

Section 8 - Bi-LSTM Model



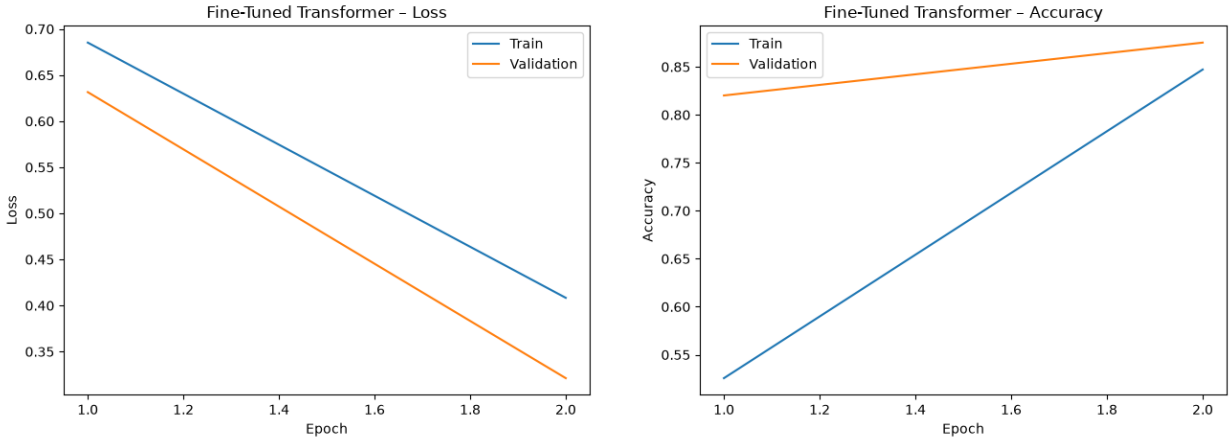
(a) Train/validation loss

(b) Train/validation accuracy

Figure 3: Section 8 - bi-LSTM with frozen Transformer embeddings.

Quantity	Value
Test loss	0.4266
Test accuracy	0.8514
Negated-polarity accuracy (62 ex.)	0.6129
Rare-words accuracy (50 ex.)	0.7800

Section 9 - Fine-Tuned Transformer Model (distilroberta)



(a) Train/validation loss

(b) Train/validation accuracy

Figure 4: Section 9 - fine-tuned distilroberta.

Quantity	Value
Test loss	0.2756
Test accuracy	0.8919
Negated-polarity accuracy (62 ex.)	0.7419
Rare-words accuracy (50 ex.)	0.7400

1 Section 10 - Comparing the Models

10.1 One-hot vs. Transformer-embedding log-linear

The two log-linear models share the same architecture and differ only in the input representation, so it is not surprising that their overall accuracy is close (test 0.8233 vs. 0.8368, validation essentially tied). Both average the word vectors and reduce the sentence to a bag of words, which caps how well either can do.

The clear difference is on the rare-words subset (0.5400 vs. 0.8400). This makes sense: the one-hot model gives every rare word its own coordinate that was barely updated during training, so it carries almost no signal. In the Transformer embedding space, on the other hand, a rare word most likely lands in the same region as more common words of similar meaning, so the sentiment signal learned for those common words generalizes to the rare one. The embedding effectively turns "rare" into "close to something familiar," and that is exactly the kind of generalization the one-hot representation cannot provide.

10.2 Adding the LSTM and the fine-tuned Transformer

Both of the more expressive models beat both log-linear models. By test accuracy the ordering is

$$\text{fine-tuned } (0.8919) > \text{bi-LSTM } (0.8514) > \text{transf-avg } (0.8368) > \text{one-hot } (0.8233).$$

The bi-LSTM improves on the log-linear models because it reads the embeddings in order (in both directions) instead of averaging them, so it can use word order and composition.

Its training curves also show a point worth noting: the model overfits in the *loss* (validation loss rises after epoch 2) but not in the *accuracy* (validation accuracy stays roughly flat). The reason is that accuracy only looks at which side of 0.5 a prediction falls on, while the BCE loss penalizes confidence. As training continues the model becomes more certain, so its mistakes turn into confidently-wrong predictions and the loss grows, even though the predictions do not actually flip sides. In other words, the model is only becoming more certain about the mistakes it was already making.

The fine-tuned Transformer is the best model by a clear margin, and it reached this on only 2,500 sentences - roughly 30 times less data than the 73,570 items the other three models saw, and without any sub-phrases. Despite that handicap it still wins, which is the strength of large-scale pretraining: the model already encodes strong general-purpose representations, so a small amount of task data is enough to adapt it. If anything its 0.8919 is a lower bound, since with the full training set we would expect it to do at least as well.

This overall ordering does not hold everywhere, though. On the rare-words subset the fine-tuned Transformer drops below the Transformer-embedding log-linear model (0.7400 vs. 0.8400) - even though both use the same pretrained distilroberta embeddings and differ only in whether those embeddings are fine-tuned or frozen. We discuss this reversal in Section 10.3.

10.3 The two special subsets

Negated polarity. The fine-tuned Transformer is clearly best (0.7419); the other three cluster together and much lower (one-hot 0.6452, transf-avg 0.6290, bi-LSTM 0.6129). Negation requires understanding that a word like "not" flips the polarity of what follows. The two averaging models cannot represent this at all, since they sum word vectors and lose the interaction between the negator and the word it negates. Only the fine-tuned Transformer, whose self-attention lets every word condition on the negator, handles negation noticeably better. The bi-LSTM is in fact the lowest of the four here, but with only 62 examples the differences among the bottom three are small and within noise.

Rare words. This is the one place the fine-tuned model does *not* win:

$$\text{transf-avg } (0.8400) > \text{bi-LSTM } (0.7800) > \text{fine-tuned } (0.7400) \gg \text{one-hot } (0.5400).$$

The one-hot model is worst by a wide margin for the reason given in Section 10.1. The interesting part is that fine-tuning actually *hurts* rare-word accuracy relative to the frozen embedding. We read this as fine-tuning destroying exactly the property that made rare words work in Section 10.1. In the frozen embedding a rare word sits next to familiar words of similar meaning, so its sentiment signal is generalizable; but when we fine-tune the whole model on only 2,500 sentences, the weights move to fit that small distribution and the rare word's neighbourhood structure is disrupted - the rare word no longer benefits from sharing latent space with the common words.

This is a known danger of fine-tuning an entire pretrained model (it tends to forget the structure it came with), and it is why for tasks like this one it is often preferable to adapt the model more

lightly: adding a LoRA adapter, fine-tuning only the last layer, or unfreezing only a randomly chosen subset of the weights. These keep most of the pretrained representation intact - including the rare-word neighbourhood structure - while still adapting the model to the task. (With only 50 examples in this subset some of the gap is also noise, but the direction is consistent with this explanation.)

2 Section 11 - Counting Parameters

Setup / notation.

- d = model dimension (width of a single token's vector).
- V = vocabulary size.
- n = context length (how many tokens are fed in at once).
- L = number of layers.
- A *parameter* is one individual learned number (one entry of a weight matrix). The parameter count is how many numbers must be stored and learned.

11.1 Transformer

(a) Token embedding matrix. The embedding is a $V \times d$ lookup table (one d-dimensional vector per vocabulary token), so it has

$$V \cdot d \quad \text{parameters.}$$

(b) Whole encoder (embedding + L attention layers). Each attention layer has four projections W_Q, W_K, W_V, W_O, each $d \times d$, giving $4d^2$ per layer and $4Ld^2$ across L layers. Adding the embedding,

$$\boxed{V \cdot d + 4Ld^2}.$$

Which part dominates? ($V = 30{,}000$, $d = 512$).

- Embedding: $Vd = 30{,}000 \times 512 \approx 15.4\text{M}$.
- Attention: $4d^2 = 4 \times 512^2 \approx 1.05\text{M}$ per layer.

The comparison reduces to V vs. $4Ld$: that is 30,000 vs. $\approx 2{,}048 \cdot L$, which are equal only around $L \approx 15$. For a typical small L the **embedding dominates**. Intuitively, the embedding carries a factor of V (30,000) in front of d, while each attention layer carries only $4d \approx 2{,}000$; the huge vocabulary outweighs the d^2 of attention unless many layers are stacked.

Does any part depend on n? No - neither Vd nor $4Ld^2$ contains n. The weights are *shared across all positions*: the embedding is one fixed $V \times d$ dictionary (a longer sequence just means more lookups in the same table, not more rows), and the four $d \times d$ projections are applied to every token with the same weights (a longer sequence reuses them more times, it does not create new ones). So n governs how many times the weights are *used* (runtime compute, and the n^2 attention map), not how many weights exist.

11.2 RNN

Replacing the L attention layers with L recurrent layers, where one recurrent layer's time-shared transformation has a fixed parameter count R (given as a black box), and keeping the embedding matrix (tokens still have to become vectors):

$$\boxed{V \cdot d + LR}.$$

Here Vd is the same embedding matrix as the Transformer, and LR is the recurrent part (R per layer $\times$ L layers).

Does it depend on n? No. R is fixed per layer regardless of sequence length: the recurrent transformation is time-shared, so the same weights are applied at every timestep and a longer sequence just means more timesteps reusing the same R parameters. This is exactly what lets an RNN handle arbitrarily long sequences with a fixed parameter budget.

11.3 Characters as tokens

Switching from word tokens to character tokens, V shrinks a lot ($\sim$30,000 $\to$ $\sim$100) but n grows ($\sim$5$\times$) to cover the same amount of text.

(a) By what factor does the embedding matrix ($V \cdot d$) shrink? d is unchanged, so only the V ratio matters:

$$30{,}000/100 = \sim\mathbf{300}\times \textbf{ smaller}.$$

The 5$\times$ growth in n does not enter here, since n is not in the embedding term.

(b) Do the attention projections / recurrent parameters change? No. $4Ld^2$ depends only on d and L, and LR depends only on L and R; none of these move when V shrinks or n grows. Since n appears in neither formula, growing it 5$\times$ changes nothing.

Combining (a) and (b): does each model's total go up or down? Down, for both models. The embedding term - which *dominated* the total in Part 1 - collapses by $\sim$300$\times$, while everything else (attention $4Ld^2$ / recurrent LR) stays flat, so the total drops substantially.

In summary, growing n by 5$\times$ costs no parameters (only runtime compute), while shrinking V saves a large amount, so character-level tokenization makes both models smaller in parameter count - the trade-off is paid in sequence length rather than in weights. And for an autoregressive model that cost is not only extra computation but also extra *time*: since such a model produces tokens one at a time, a $\sim$5$\times$ longer sequence means $\sim$5$\times$ more sequential generation steps that cannot be parallelized away, on top of the higher per-step compute (such as the n^2 attention) that the longer context adds anyway.

AI usage declaration

We used Claude Code to assist with writing and debugging parts of the code.